

ARCHITECTURE AND OPERATIONS DOCUMENTATION

AI Customer Operations Platform

High-level design, detailed design, network and data flows, technology stack, and deployment guide.

GENERATED FROM THE REPOSITORY DOCUMENTATION

High-Level Design

1. Purpose

The AI Customer Operations Platform is a multi-tenant SaaS application for managing customer contacts, conversations, appointments, business settings, and WhatsApp operations. The first vertical is clinics, but the foundation uses generic organizations, contacts, services-ready configuration, conversations, and appointments so future verticals do not require a platform rewrite.

This document describes the current implemented design. It does not describe unimplemented AI automation, billing, or future microservices.

2. Goals

- Provide a secure organization and membership boundary.
- Support authenticated operators through a protected dashboard.
- Store contacts, conversations, messages, appointments, and business configuration.
- Receive and send WhatsApp Cloud API messages through a controlled server boundary.
- Preserve message idempotency, delivery status, retry state, and tenant isolation.
- Prepare deterministic AI scheduling actions without allowing a model to authorize mutations.
- Keep the system deployable as one modular monolith.
- Make future domain modules addable without weakening RLS or server/client boundaries.

3. Non-goals in the current release

- No billing or subscription management.
- No automatic natural-language reply generation.
- No WhatsApp template-message workflow.
- No escalation-rule engine.
- No service, staff, resource, or clinic-specific scheduling model beyond the implemented appointment domain.
- No exactly-once provider delivery guarantee; ambiguous provider outcomes are marked `unconfirmed`.
- No in-process rate limiter; rate limiting belongs at the deployment edge.
- No microservices, Kafka, Redis, or Kubernetes.

4. Architectural style

The system is a **modular monolith**:

- One Next.js deployable contains the UI, server components, server actions, and route handlers.
- Supabase PostgreSQL is the durable system of record.
- Supabase Auth owns user identity and sessions.
- PostgreSQL RLS is the authoritative tenant authorization boundary.
- Server-only modules own privileged provider and service-role operations.
- Domain repositories and services isolate business rules from pages and transport adapters.

flowchart LR
User[Operator browser] -->|HTTPS and cookies| Web[Next.js application]
Meta[Meta WhatsApp Cloud API] -->|Verified webhook| Web
Web -->|SSR anon client and RLS| DB[(Supabase PostgreSQL)]
Web -->|Auth session| Auth[Supabase Auth]
Web -->|Vault-backed provider call| Meta
Web -->|Server-only classification| OpenAI[OpenAI API]
Cron[Supabase pg_cron and pg_net] -->|Worker secret| Web
Vault[Supabase Vault] -->|Service-only secret resolution| Web

5. Major components

5.1 Web application

The `app/` directory contains the App Router pages and route handlers. It provides:

- Public landing and authentication pages.
- Onboarding and organization selection.
- Protected dashboard pages for contacts, conversations, appointments, and settings.
- `/api/health` for application liveness.
- `/api/whatsapp/webhook` for Meta verification and inbound/status events.
- `/api/internal/whatsapp/retry` for the authenticated durable retry worker.

5.2 Authentication and tenant context

Supabase Auth authenticates the user. `organization_members` authorizes access to an organization. Server-side organization context revalidates membership on every request and does not treat a client organization ID or preference cookie as proof of access.

Roles are `owner`, `admin`, and `member`. Database policies and server-side checks enforce role-sensitive writes.

5.3 Domain layer

`lib/domain/` contains organization-scoped repositories, validation, typed errors, business configuration, contacts, conversations, messages, and appointments. Repositories use the authenticated SSR client and explicit allowed fields. Appointment services own availability, time zone, conflict, status, and concurrency rules.

5.4 WhatsApp integration

`lib/whatsapp/` contains configuration resolution, Meta request handling, signature validation, inbound normalization, outbound sending, delivery reconciliation, failure classification, retries, and pipeline orchestration.

Provider access tokens, app secrets, and verify tokens are resolved from Supabase Vault through service-only functions. They are never accepted from a browser request or returned to a client.

5.5 AI receptionist core

`lib/ai/` contains a server-only provider boundary, strict intent classification, derived conversation state, scheduling conversation planning, extraction, and deterministic appointment tools. The model may classify or extract data, but it cannot select a tenant, provide an appointment identifier, call a tool, or mutate the database.

5.6 Database and operations

Supabase PostgreSQL stores all durable records and enforces RLS, constraints, foreign keys, and security-definer boundaries. `whatsapp_send_jobs` supports durable retry. `pg_cron` and `pg_net` invoke the internal retry endpoint when production Vault configuration is present.

6. Primary user journeys

Operator journey

1. User signs up or logs in through Supabase Auth.
2. Middleware and server context validate the session.
3. User creates or selects an authorized organization.
4. User operates contacts, conversations, appointments, and business settings in the protected dashboard.
5. RLS prevents access to another organization's records.

Inbound WhatsApp journey

1. Meta sends a webhook request.
2. GET verification resolves the tenant configuration and validates the challenge.
3. POST extracts only a routing hint, resolves configuration server-side, verifies the raw-body HMAC, and acknowledges invalid signatures with `403`.
4. A verified inbound text event is normalized.
5. The pipeline resolves or creates the organization-scoped contact and open conversation.
6. The inbound message is inserted once using provider-message idempotency.
7. AI behavior may derive state and a scheduling plan, but the current release stops at structured tool results and does not generate a reply.

Outbound and retry journey

1. A trusted server operation reserves an outbound message row.
2. The provider request is sent using Vault-resolved configuration.
3. The result is correlated to the existing message row.
4. Safe retryable failures create one durable send job.
5. The worker claims jobs with a lease and `SKIP LOCKED`, retries with bounded jittered backoff, or retires a job as `dead`.
6. Ambiguous outcomes remain `unconfirmed` and are not retried automatically.

7. Quality attributes

- **Security:** RLS, strict server/client separation, HMAC webhook validation, timing-safe worker authentication, safe errors, fixed search paths, and secret isolation.
- **Reliability:** idempotent inbound persistence, monotonic delivery states, durable retry jobs, lease recovery, bounded attempts, and explicit ambiguous outcomes.
- **Scalability:** stateless Next.js instances, managed PostgreSQL, and tenant-scoped indexes; edge rate limiting is required before high-volume production use.
- **Maintainability:** TypeScript strict mode, modular domains, Zod contracts, migrations, unit/integration/E2E tests, and CI.
- **Observability:** health endpoint, application logs, delivery/job state inspection, and deployment verification runbook. A full audit/event platform is future work.

8. Key decisions

Decision	Reason
Modular monolith	Keeps the current product simple while preserving domain boundaries.
Supabase Auth plus PostgreSQL RLS	Identity and tenant authorization are separate, database-enforced concerns.
Service role only for trusted WhatsApp paths	Prevents browser or ordinary tenant queries from bypassing RLS.
Vault for provider secrets	Keeps tokens and signing secrets out of code, client payloads, and environment variables.
Deterministic AI tool selection	A model cannot directly authorize an appointment mutation.
<code>unconfirmed</code> for ambiguous sends	The WhatsApp text API does not expose a client idempotency key, so automatic retry could duplicate a customer message.
No application rate limiter	In-memory limits are unsafe across instances; use an edge/shared limiter at deployment.

Detailed Low-Level Design

1. Repository and module layout

app/	
api/health/	liveness endpoint
api/whatsapp/webhook/	Meta verification and event ingress
api/internal/whatsapp/retry/	authenticated retry worker ingress
auth/	callback, login, logout, signup
dashboard/	protected operator UI
components/	client forms and navigation
lib/	
auth/	auth actions and validation
config/	environment contract
domain/	tenant-scoped repositories and services
appointments/	appointment engine and rules
business/	business configuration repository
contacts/	contact repository
conversations/	conversation repository
messages/	message repository
organizations/	membership and organization context
supabase/	browser, SSR, service/database clients
ai/	classification, state, planning, tools
whatsapp/	provider adapter and reliability pipeline
types/	generated and shared TypeScript types
supabase/migrations/	ordered PostgreSQL schema and policy changes
tests/	unit, integration/RLS, and Playwright E2E

Import rules

- Browser components may use the browser Supabase client only.
- Server components, actions, and route handlers use the SSR client.
- `server-only` modules cannot be imported into client components.
- Provider secrets and the service-role key are confined to server-only WhatsApp paths.
- AI modules do not import Supabase or WhatsApp modules for classification/state/planning.
- AI tools delegate to the authoritative appointment operations; they do not duplicate scheduling rules.

2. Runtime layers

flowchart TB
UI[Client components and Server Components] --> Transport[Pages, Server Actions, Route Handlers]
Transport --> Context[Auth and organization context]
Context --> Domain[Repositories and domain services]
Domain --> Integration[WhatsApp and AI adapters]
Integration --> Data[(Supabase PostgreSQL)]
Data --> External[Meta and OpenAI]
UI --> Transport
Transport --> Context
Context --> Domain
Domain --> Integration
Integration --> Data
Data --> External

3. Database model

The exact schema is defined by ordered migrations in `supabase/migrations/`. The logical model is:

erDiagram AUTH_USERS ||--o| PROFILES : has AUTH_USERS ||--o{ ORGANIZATION_MEMBERS : joins ORGANIZATIONS ||--o{ ORGANIZATION_MEMBERS : contains ORGANIZATIONS ||--o{ CONTACTS : owns CONTACTS ||--o{ CONVERSATIONS : has CONVERSATIONS ||--o{ MESSAGES : contains ORGANIZATIONS ||--o{ APPOINTMENTS : owns CONTACTS ||--o{ APPOINTMENTS : books CONVERSATIONS ||--o{ APPOINTMENTS : optionally_contextualizes ORGANIZATIONS ||--o| BUSINESS_PROFILE : configures ORGANIZATIONS ||--o| SCHEDULING_SETTINGS : configures ORGANIZATIONS ||--o{ BLOCKED_PERIODS : defines ORGANIZATIONS ||--o| WHATSAPP_CONFIG : configures ORGANIZATIONS ||--o| RECEPTIONIST_SETTINGS : configures MESSAGES ||--o| WHATSAPP_SEND_JOBS : retries

Core records

- `profiles` : application profile linked to `auth.users`.
- `organizations` : tenant identity, slug, and business profile fields.
- `organization_members` : user membership and constrained role.
- `contacts` : organization-owned customer identities, including phone numbers.
- `conversations` : organization/contact/channel conversation state.
- `messages` : inbound/outbound content, provider correlation, and delivery state.
- `appointments` : contact-linked appointment lifecycle with UTC instants.
- `organization_scheduling_settings` : business hours, IANA timezone, and default duration.
- `organization_blocked_periods` : organization-owned unavailable time ranges.
- `organization_whatsapp_configs` : safe provider metadata only.
- `organization_whatsapp_secret_refs` : Vault references, inaccessible to client roles.
- `organization_receptionist_settings` : bounded tenant-authored instructions and FAQ.
- `whatsapp_send_jobs` : durable retry state, lease, attempt count, and next-attempt time.

Every organization-owned row has `organization_id` and same-tenant composite relationships where required. Timestamps are `timestampz` and database-generated.

4. Authorization design

1. Supabase Auth establishes the authenticated user.
2. Organization context reads memberships using the SSR anon client.
3. The selected organization is a preference only; membership is revalidated.
4. Repositories derive organization context and do not accept caller tenant IDs.
5. PostgreSQL RLS checks membership for reads and role-sensitive mutations.
6. Server-side role checks provide an additional application boundary.
7. Service-role operations are limited to trusted WhatsApp correlation/retry paths and never used as a substitute for tenant authorization.

RLS is enabled on all public tables. `organization_whatsapp_secret_refs` and `whatsapp_send_jobs` intentionally have no `anon` or `authenticated` policies.

5. HTTP and route contracts

Health

`GET /api/health`

- Returns application liveness only.
- Does not call the database, Meta, or OpenAI.

- Use it for deployment/load-balancer checks, not dependency readiness.

WhatsApp webhook

GET /api/whatsapp/webhook

- Resolves configuration from the verified routing boundary.
- Validates the Meta verify token and returns the challenge.
- Does not expose Vault data.

POST /api/whatsapp/webhook

- Reads the raw request body.
- Extracts only an untrusted phone-number routing hint.
- Resolves active configuration server-side.
- Verifies `X-Hub-Signature-256` HMAC before parsing/persisting.
- Persists verified inbound text and status events through the pipeline.
- Returns `403` for invalid signatures and `500` for persistence failures.
- Acknowledges verified duplicates, stale events, and unknown events deterministically.

Retry worker

POST /api/internal/whatsapp/retry

- Requires a bearer secret compared with a timing-safe comparison.
- Accepts no tenant, provider, or message identity from the caller.
- Claims due jobs server-side and returns aggregate processing information without internal error detail.
- Is disabled when `WHATSAPP_RETRY_WORKER_SECRET` is unset.

6. Appointment engine contract

The appointment engine is the sole owner of appointment mutations:

- Create validates local date/time, converts to UTC using organization timezone, checks business hours, blocked periods, duration, and conflicts.
- Reschedule revalidates ownership and handles self-conflict correctly.
- Cancellation uses a dedicated transition boundary.
- Query is organization/contact scoped and returns bounded mapped records.
- Status transitions prevent invalid terminal-state changes.
- AI tools call these operations rather than writing appointment rows directly.

The current model has no staff, service, or resource selection. Those concepts must be added through a reviewed domain change, not guessed in the AI layer.

7. WhatsApp reliability design

Inbound

- Contact resolution is organization scoped.
- One open WhatsApp conversation exists per organization/contact/configuration.
- Provider message ID has tenant-scoped uniqueness.
- Duplicate events return a stable duplicate result and do not insert a second message.

- Concurrent duplicates converge through transaction locking and uniqueness constraints.

Delivery states

Outbound messages use a monotonic state machine:

```
pending -> sent -> delivered -> read
pending -> failed
sent -> failed
```

- Duplicate callbacks are ignored.
- Regressions are ignored as stale.
- A delivered/read message cannot become failed.
- Inbound messages do not receive outbound delivery status.

Retry jobs

- States: `pending`, `processing`, `completed`, `dead`.
- Claim uses `FOR UPDATE SKIP LOCKED` and a five-minute lease.
- Claim increments attempts before send, including worker-crash attempts.
- Five attempts maximum; exponential full-jitter backoff, 30-second base, one-hour cap.
- HTTP 429 honors the greater of `Retry-After` and calculated backoff, capped at one hour.
- Retryable failures create one job; permanent failures create none.
- Ambiguous outcomes become `unconfirmed` and are not automatically retried.

8. AI receptionist design

Classification

- Seven intents: booking, rescheduling, cancellation, query, general question, greeting, unknown.
- Strict Zod parsing rejects unsupported values and extra keys.
- Provider failure, malformed output, low-signal input, and unknown intent require clarification.
- The customer message is delimited untrusted data; the model cannot override system instructions or use tools.

State and planning

- State is derived from the authenticated organization, conversation, contact, and latest inbound message.
- History is bounded to the latest 20 messages in chronological order.
- The latest inbound message is classified even if outbound messages are newer.
- Scheduling planning extracts only local date/time and safe appointment context references.
- Timezone comes from organization scheduling settings; DST ambiguity/nonexistence requests clarification.
- Planning is read-only.

Tool execution

- Exactly four tools: book, reschedule, cancel, query.
- Tool choice is deterministic from the validated plan; the model is not involved in execution.
- Organization and contact are derived server-side.

- Appointment references resolve against the trusted contact's pending/confirmed records by local date/time; zero or multiple matches do not mutate.
- Results are discriminated and expose only safe domain codes.
- No reply generation or webhook wiring is currently performed by the AI layer.

9. Error handling

- Zod validation failures map to stable validation responses.
- Database/provider details are translated into typed domain or integration errors.
- Public responses contain safe error codes and generic messages.
- Logs may contain operational context but must not contain credentials, Vault values, raw provider secrets, or untrusted full payloads unless explicitly scrubbed.
- Retry classification distinguishes retryable, ambiguous, and permanent failures.

10. Test layers

- Unit tests cover validation, intent contracts, state/planning/tool behavior, retry policy, and static security boundaries.
- Integration tests cover Supabase schema, RLS, domain behavior, WhatsApp pipeline/reliability, AI state/planning/tools, and business configuration.
- Playwright covers browser workflows such as authentication, dashboard, contacts, conversations, and appointments.
- CI runs lint, typecheck, unit tests, build, and a fresh local Supabase migration/RLS integration job.

Network Architecture and Data Flow

1. Scope

This document describes the network and trust boundaries for the current modular monolith. The selected hosting platform is intentionally not hard-coded; the application can run on a Next.js-compatible host with Supabase as the managed data platform.

2. Logical topology

flowchart TB Browser[Operator browser\nHTTPS] --> Edge[DNS / TLS / hosting edge\nrate limiting required here] Meta[Meta Graph API\nWhatsApp Cloud API] -->|HTTPS webhook| Edge Cron[Supabase pg_cron + pg_net] -->|HTTPS POST + bearer| Edge Edge --> App[Next.js application\nstateless instances] App -->|SSR anon key + user cookies| Supabase[Supabase API / PostgREST] App -->|Auth SDK| SupabaseAuth[Supabase Auth] App -->|service-only Vault RPC| Vault[Supabase Vault] App -->|HTTPS server-only API key| OpenAI[OpenAI API] App -->|HTTPS provider token| Meta Supabase --> Postgres[(Supabase PostgreSQL)] Vault --> Postgres

3. Trust zones

Zone	Components	Trust level	Controls
Public/untrusted	Browser, Meta webhook payload, customer message text	Untrusted	HTTPS, input validation, HMAC verification, React escaping, no caller tenant IDs.
Application	Next.js pages, actions, route handlers	Controlled	Auth/session checks, server-only imports, typed contracts, safe errors.
Tenant data	Supabase PostgreSQL tables	Sensitive	RLS, organization-scoped queries, constraints, same-tenant foreign keys.
Privileged operations	Service-role client, Vault functions, retry worker	Highly restricted	Server-only modules, no public execute grants, timing-safe worker secret, narrow procedures.
External providers	Meta and OpenAI	Third-party	Explicit request schemas, no secret leakage, failure classification, response reduction.

4. Network rules and ports

- Public application traffic: TCP 443 only in production.
- Local Next.js development: TCP 3000 by default.
- Supabase services are accessed over HTTPS through the Supabase project URL.
- OpenAI and Meta are accessed only by server-side code over HTTPS.
- No direct browser-to-Meta provider request is allowed for application messaging.
- No database port is exposed by the application; database access is through Supabase clients/functions.
- The internal retry endpoint is still HTTPS-public at the transport layer, but requires a high-entropy bearer secret and does not accept tenant selection input.
- Configure platform edge rate limiting before high-volume production traffic. The application intentionally does not use process-local rate limiting.

5. Browser request flow

sequenceDiagram participant B as Browser participant N as Next.js participant A as Supabase Auth participant P as Supabase PostgREST B->>N: HTTPS page/action request with session cookie N->>A: Read/refresh authenticated session A-->>N: User identity N->>P: SSR request with user session and anon key P->>P: Evaluate RLS using auth.uid() P-->>N: Tenant-scoped rows or safe error N-->>B: Rendered page or safe response

The browser may hold the public Supabase URL and anon key, but those values do not grant access. RLS and authenticated session claims are the authorization boundary.

6. Inbound WhatsApp data flow

sequenceDiagram participant M as Meta participant W as Webhook route participant V as Vault/config boundary participant P as Pipeline participant D as PostgreSQL participant AI as AI planning modules M->>W: GET verification challenge W->>V: Resolve active config by routing boundary V-->>W: Verify-token reference/value inside server boundary W-->>M: Challenge or rejection M->>W: POST raw event + X-Hub-Signature-256 W->>W: Extract untrusted phone hint W->>V: Resolve organization config server-side V-->>W: App secret/config metadata W->>W: Verify HMAC over raw body alt invalid signature W-->>M: 403; no persistence else valid signature W->>P: Normalize trusted event P->>D: Resolve/create tenant-scoped contact P->>D: Resolve/create open conversation P->>D: Insert message under provider-id uniqueness D-->>P: Inserted or stable duplicate result P-->>W: Deterministic acknowledgement W->>AI: Optional state/planning/tool boundary AI-->>W: Structured result only; no generated reply W-->>M: 200 acknowledgement end

Input trust rules

- The phone-number routing hint is never treated as authorization.
- Raw body signature verification occurs before trusting the event payload.
- Customer text is untrusted data for AI classification and extraction.
- Provider message IDs are correlation keys, not tenant selectors.

7. Outbound WhatsApp data flow

sequenceDiagram participant S as Trusted server operation participant D as PostgreSQL participant V as Vault participant W as WhatsApp module participant M as Meta S->>D: Reserve outbound message in tenant context W->>V: Resolve active config and access token V-->>W: Provider secret inside server boundary W->>M: HTTPS text message request alt accepted and correlated M-->>W: Provider message ID W->>D: Correlate existing row; set sent else retryable failure W->>D: Set failure classification W->>D: Create/reuse one retry job else ambiguous outcome W->>D: Set unconfirmed; no automatic retry else permanent failure W->>D: Record failed; no retry job end

Meta status callbacks later advance `sent -> delivered -> read` or apply `failed` only before delivery. Stale and duplicate callbacks are ignored.

8. Retry-worker data flow

```
sequenceDiagram
    participant C as pg_cron / pg_net
    participant R as Retry route
    participant D as PostgreSQL
    participant V as Vault
    participant M as Meta
    C->>V: Read worker URL and worker secret
    V-->>C: Runtime values or absent
    alt both values absent
    C->>C: Remain inert else configured
    C->>R: HTTPS POST with bearer secret
    R->>R: Timing-safe secret comparison
    R->>D: Claim due job FOR UPDATE SKIP LOCKED
    D-->>R: Lease and increment attempt
    R->>V: Resolve tenant provider configuration
    R->>M: Retry existing logical message
    M-->>R: Success or classified failure
    R->>D: Complete, reschedule, or dead-letter job
    R-->>C: Safe aggregate response
    end
```

A retry never creates a second logical message row. A worker crash consumes an attempt because attempts increment at claim time.

9. AI data flow and minimization

1. The latest inbound message is selected from a bounded 20-message conversation window.
2. Only the required text and bounded conversation state are given to the server-only AI boundary.
3. The model receives a classification/extraction prompt with customer content delimited as untrusted data.
4. Strict Zod parsing reduces the response to a known internal contract.
5. Provider errors, malformed output, and low-signal text become clarification/unknown outcomes.
6. Tool arguments are derived from trusted conversation/contact state and validated again.
7. Appointment tools use existing domain services; raw rows and infrastructure errors are not exposed.

Business instructions and FAQ are stored as tenant-authored data but are not yet injected into AI prompts.

10. Data classification and retention guidance

Data	Classification	Allowed locations
Auth session cookie	Sensitive	Secure HTTP-only cookie and Supabase Auth.
Supabase URL/anon key	Public configuration	Browser and server environment. RLS remains mandatory.
Service-role key	Secret	Server deployment secret only.
Meta access token/app secret/verify token	Secret	Supabase Vault only.
Worker secret	Secret	Deployment secret and matching Vault entry.
Customer phone/message content	Tenant-sensitive	Tenant-scoped PostgreSQL rows and necessary provider/AI requests.
Appointment and business settings	Tenant-sensitive	Tenant-scoped PostgreSQL rows.
Provider message ID/status	Tenant-sensitive operational data	Tenant-scoped message/reliability records.

Do not log secrets, raw authorization headers, Vault contents, complete webhook bodies, or unnecessary customer content. Configure provider and platform retention according to the production data policy before launch.

11. Failure boundaries

- Invalid webhook signature: reject at the application edge before persistence.

- Unknown/stale/duplicate verified event: acknowledge safely and do not repeat side effects.
- Database persistence failure: return `500` so the provider can retry; never misreport it as a signature failure.
- Provider timeout/connect failure: classify as retryable or ambiguous based on whether acceptance is known.
- OpenAI unavailable: return an unknown/clarification result; do not mutate appointments.
- Unauthorized tenant access: RLS or server context rejects the operation.
- Missing retry Vault configuration: cron invoker is intentionally inert.

Technology Stack

1. Application stack

Area	Technology	Current use
Language	TypeScript, strict mode	Application, domain, tests, generated database types.
Web framework	Next.js 16.3.1	App Router, Server Components, Route Handlers, middleware, production build.
UI runtime	React 18.3.1	Server-rendered pages and client forms/components.
Styling	Tailwind CSS 3.4.x	Utility styling and responsive dashboard UI.
Components	shadcn/ui foundation and Lucide icons	Reusable interface components and icons.
Validation	Zod 4	Request, domain, AI output, and environment contracts.
Auth	Supabase Auth with <code>@supabase/ssr</code>	Session handling, cookies, callback flow, user identity.
Database	Supabase PostgreSQL	Tenant data, constraints, RLS, RPCs, Vault integration, cron substrate.
Database client	<code>@supabase/supabase-js</code>	Browser, SSR, and restricted service-side access.
Server boundary	<code>server-only</code>	Prevents secrets and privileged modules entering client bundles.
Provider	Meta WhatsApp Cloud API	Webhook, text sending, and delivery status callbacks.
AI provider	OpenAI-compatible server API	Intent classification and bounded extraction; optional by environment.
Unit/integration tests	Vitest	Fast contracts, domain, security, pipeline, reliability, and integration suites.
Browser tests	Playwright	End-to-end authenticated/operator workflows.
Static quality	ESLint, Prettier, TypeScript	Lint, formatting, and compile-time quality gates.
CI/CD	GitHub Actions	Build/test job and fresh local Supabase integration/RLS job.
Package manager	npm with lockfile	Reproducible installs via <code>npm ci</code> .
Hosting	Next.js-compatible deployment platform	Platform choice remains deployment-specific.

2. Runtime topology

- **Browser:** receives public application code and public Supabase configuration only.
- **Next.js runtime:** executes pages, actions, route handlers, auth context, domain services, and provider calls.
- **Supabase:** provides Auth, PostgreSQL, RLS, Vault, `pg_cron`, and `pg_net`.
- **External providers:** Meta receives signed webhook responses and outbound API requests; OpenAI receives only the bounded AI request when configured.

3. Dependency rules

- Prefer existing platform APIs and local modules before adding a dependency.
- Keep provider adapters behind server-only boundaries.
- Use Zod for external and domain contracts.
- Do not introduce an ORM, second database, message broker, Redis, or microservice without an approved architecture change.
- Keep generated database types synchronized with migrations.
- Never use `any`, `@ts-ignore`, or client-provided tenant IDs for authorization.

4. Environment configuration

Public and required

- `NEXT_PUBLIC_SUPABASE_URL`
- `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- `NEXT_PUBLIC_SITE_URL`

Server-only and required in production

- `SUPABASE_SERVICE_ROLE_KEY`

Server-only and optional

- `OPENAI_API_KEY`: absent means intent is `unknown` and clarification is required.
- `OPENAI_INTENT_MODEL`: optional model override.
- `WHATSAPP_RETRY_WORKER_SECRET`: absent disables the internal retry route.

Test-only

- `SUPABASE_TEST_URL`
- `SUPABASE_TEST_ANON_KEY`
- `SUPABASE_TEST_SERVICE_ROLE_KEY`

WhatsApp access tokens, app secrets, verify tokens, retry-worker URL, and the Vault copy of the worker secret are managed in Supabase Vault and are not environment variables or client fields.

Validate the contract with:

```
npm run verify:env
```

The command prints names and warnings, never secret values.

5. Security technology controls

- PostgreSQL RLS on all public tables.
- No client policies on secret references and retry jobs.
- Fixed `search_path` on `SECURITY DEFINER` functions.
- No public execute grant on privileged functions.
- HMAC validation for Meta webhook requests.
- Timing-safe bearer comparison for the retry worker.

- Strict security headers and `poweredByHeader: false`.
- HTTPS and secure cookies in production.
- Zod validation and typed safe error mapping.
- No service-role key in browser code.

CSP is intentionally not configured until the hosting platform's per-request nonce strategy is selected. Edge rate limiting is also a deployment requirement, not an in-process application feature.

6. Quality gates

```
npm run lint
npm run typecheck
npm test -- --run
npm run build
npm run test:integration
npm run test:e2e
```

CI automatically runs lint, typecheck, unit tests, build, and a fresh local Supabase migration plus integration/RLS suite. E2E execution may require the configured browser environment and test data.

7. Version and compatibility notes

- Use the Node.js version defined by CI, currently Node 22.x.
- Install with `npm ci` so `package-lock.json` remains authoritative.
- Review the repository's `AGENTS.md` and the installed Next.js documentation before changing Next.js APIs.
- Apply migrations in timestamp order.
- Regenerate Supabase TypeScript types after schema changes.

Deployment Guide

1. Deployment model

The application is a Next.js modular monolith deployed to a Next.js-compatible host. Supabase is a separate managed dependency providing PostgreSQL, Auth, Vault, `pg_cron`, and `pg_net`. Meta WhatsApp Cloud API and OpenAI are external provider dependencies.

The exact hosting provider is not fixed in the repository. The host must support:

- Node.js 22.x or a compatible current runtime.
- Next.js production build and `next start` /platform equivalent.
- HTTPS with a stable public URL.
- Server-only environment secrets.
- Long enough request handling for webhook and provider boundaries.
- Platform or WAF rate limiting before production traffic.

2. Prerequisites

- Node.js 22.x for parity with CI.
- npm and Git.
- A Supabase project for deployment.
- Supabase CLI for migration management, installed separately if needed.
- Meta Developer App and WhatsApp Cloud API configuration for live integration.
- OpenAI API key only if live intent classification is enabled.
- A deployment host with DNS and TLS.

3. Local development

```
git clone https://github.com/sundeepbiradar-ai/whatsapp-ai-receptionist.git
cd whatsapp-ai-receptionist
npm ci
cp .env.example .env.local
```

Set at minimum:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
NEXT_PUBLIC_SITE_URL=http://localhost:3000
```

For server paths, set a non-production service-role key only in the ignored local environment. Do not use production credentials locally.

Start the application:

```
npm run dev
```

Open `http://localhost:3000`.

Check the environment contract:

```
npm run verify:env
```

4. Pre-deployment verification

Run locally or in CI:

```
npm ci
npm run verify:env
npm run lint
npm run typecheck
npm test -- --run
npm run build
```

Run integration tests against a disposable Supabase project or local Supabase:

```
npm run test:integration
```

Run browser tests when the required browser/test environment is available:

```
npm run test:e2e
```

Do not proceed with a failed typecheck, lint, unit suite, integration suite, or production build.

5. Supabase production setup

5.1 Apply schema

From the repository root:

```
supabase login
supabase link --project-ref <production-project-ref>
supabase db push
```

For a new disposable environment, verify the full migration chain from scratch. Never run `db reset` against production.

5.2 Confirm database security

Verify in the Supabase dashboard or SQL checks:

- RLS is enabled on every public table.
- `organization_whatsapp_secret_refs` has no `anon` / `authenticated` policies.
- `whatsapp_send_jobs` has no `anon` / `authenticated` policies.
- Security-definer functions have fixed search paths and restricted execute grants.
- Vault is available and service-only Vault functions are present.
- Foreign keys, unique provider IDs, and retry-job constraints exist.

5.3 Configure Supabase services

- Enable `pg_cron` and confirm `pg_net` is available.
- Confirm the `whatsapp-retry-worker` cron job exists and runs every minute.
- Create Vault secrets per tenant for provider access token, app secret, and verify token references.

- Create Vault entries named `whatsapp_retry_worker_url` and `whatsapp_retry_worker_secret` after the application URL is known.
- The cron invoker remains intentionally inert until both worker entries exist.

If the schema changed, regenerate types:

```
supabase gen types typescript --local > lib/supabase/database.ts
```

Use the production project endpoint instead of `--local` when generating from a reviewed non-production/production schema according to the team's credential policy.

6. Application environment

Configure these in the hosting platform, never in source control:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
NEXT_PUBLIC_SITE_URL=https://your-production-domain.example
SUPABASE_SERVICE_ROLE_KEY=<server-only-service-role-key>
OPENAI_API_KEY=<optional-server-only-key>
OPENAI_INTENT_MODEL=<optional-model-name>
WHATSAPP_RETRY_WORKER_SECRET=<server-only-worker-secret>
```

Do not configure `SUPABASE_TEST_*` variables in production. Do not put Meta access tokens, app secrets, or verify tokens in environment variables; use Vault.

Run the deploy gate in the deployment environment:

```
npm run verify:env
```

The command must report no missing required variables. Warnings for intentionally disabled OpenAI or retry features must be understood and approved.

7. Build and release

The host should execute:

```
npm ci
npm run verify:env
npm run lint
npm run typecheck
npm test -- --run
npm run build
```

Deploy the generated Next.js application using the host's supported production command. For a generic Node host:

```
npm start
```

Set the public DNS record to the host, enable TLS, and set `NEXT_PUBLIC_SITE_URL` to the final HTTPS origin. Authentication callbacks depend on this value.

8. Meta WhatsApp configuration

After the application has a stable HTTPS URL:

1. Configure the Meta webhook callback as `https://<domain>/api/whatsapp/webhook`.
2. Complete GET verification with the tenant's Vault-backed verify token.
3. Configure the organization WhatsApp metadata: phone number ID, business account ID, display number, and active state.
4. Store the access token and app secret through the approved Vault references.
5. Subscribe the Meta app to inbound message and message-status events.
6. Send a controlled inbound test message.
7. Verify the contact, open conversation, inbound message, provider ID, and duplicate-event behavior.
8. Send a controlled outbound test through the trusted application path.
9. Verify delivery callbacks advance status monotonically.

9. Retry worker configuration

After deployment, set Vault values:

- `whatsapp_retry_worker_url` : `https://<domain>/api/internal/whatsapp/retry`
- `whatsapp_retry_worker_secret` : the same high-entropy value as `WHATSAPP_RETRY_WORKER_SECRET`

Then verify:

- A request without the bearer secret is rejected.
- A correct worker request claims only due jobs.
- A retryable provider failure creates/reschedules one job.
- A permanent failure creates no job.
- A spent job becomes `dead`.
- An ambiguous provider result remains `unconfirmed` and is not retried.
- `cron.job` shows the expected schedule and recent executions.

10. Smoke tests after release

```
curl -fsS https://<domain>/api/health
```

Then manually verify:

- Sign up and log in.
- Complete onboarding and load the protected dashboard.
- Create/read a contact.
- Create and update business settings as owner/admin.
- Confirm a member can read permitted settings but cannot perform restricted writes.
- Create, reschedule, cancel, and query an appointment in the configured timezone.
- Confirm two tenants cannot read or mutate one another's records.
- Verify the Meta webhook GET challenge and one inbound message.
- Verify status callbacks and retry behavior if production credentials are enabled.
- Verify live OpenAI classification only if `OPENAI_API_KEY` is configured.

11. Monitoring and operations

Monitor:

- Application error logs and request latency.
- `/api/health` availability.
- Authentication callback failures.
- PostgreSQL errors and RLS denials.
- `messages.delivery_status` transitions.
- `messages` rows with `unconfirmed` status.
- `whatsapp_send_jobs` rows in `processing`, `pending`, and `dead` states.
- Cron invocation failures and Vault resolution failures.
- Meta webhook verification/signature failures.

Do not log authorization headers, provider tokens, Vault values, raw webhook signatures, or unnecessary customer message content.

12. Backup and recovery

Use Supabase managed backups and enable point-in-time recovery before launch. Agree retention with the product owner and rehearse restoration into a non-production project. The application does not implement its own backup system.

13. Rollback

1. Stop or disable the deployment rollout at the hosting platform.
2. Redeploy the previous known-good application build.
3. Do not reverse migrations casually; repository migrations are additive and require a reviewed down-migration if reversal is necessary.
4. If the retry worker misbehaves, remove or clear the Vault `whatsapp_retry_worker_secret` so the cron invoker becomes inert without a code rollback.
5. Re-run `/api/health`, authentication, RLS, and controlled provider smoke tests.
6. Preserve failed job and message records for investigation.

14. Launch blockers and explicit verification status

The implementation is hardening-complete, but launch verification remains pending for:

- Live Meta GET webhook verification.
- Live Meta outbound delivery.
- Live Meta status callbacks.
- Hosted `pg_cron` calling the deployed worker.
- Production creation of retry-worker Vault secrets.
- Live OpenAI classification.
- Production two-tenant RLS smoke test.
- Hosting-platform CSP nonce strategy.
- Edge/WAF rate limiting.
- Backup restore rehearsal.

These items must be checked off before declaring a production launch complete.